LLM-Guided Grammar-Aware Fuzzing of PostgreSQL

CS-5187 Term Project A3 – Meeting presentation

Venu, Bhuvan, Gowtham

16 October 2026

Problem Statement

- PostgreSQL ships **new optimiser and executor features every year**; each one adds code paths that the regression suite covers only partially.
- The worst bugs are **silent wrong results**: no crash, no error, just wrong rows.
- Existing SQL fuzzers either produce mostly **invalid SQL** (grammar-only, type-blind) or use **hand-written generators** that lag behind new features.

Question: can a *typed, editable grammar*, extended by an *LLM* that reads PostgreSQL's own commits, find more bugs than random mutation and a plain grammar, under the same budget?

Motivation: why PostgreSQL, why now

- Used everywhere; a wrong-result bug can corrupt business data silently.
- Recent releases changed the planner heavily (to verify per release notes): OR-to-ANY transformation, self-join elimination, B-tree skip scan, virtual generated columns, `RETURNING OLD/NEW`, `MERGE` extensions, `JSON_TABLE`.
- `master` (20devel) gets new commits daily; regressions found there are fixed quickly and credited ("Reported-by").
- LLMs can read commit messages, docs and regression tests – exactly the knowledge a fuzzer lacks about brand-new features.

Limits of existing approaches

Approach	Strength	Gap
Random / mutation (AFL-style)	cheap, fast	mostly syntax errors
SQLsmith	type-aware, finds crashes	no wrong-result oracle
SQLancer (TLP, NoREC, PQS)	logic-bug oracles	hand-written generators
Squirrel, SQLRight	coverage-guided IR mutation	fixed grammar, slow to add features
Fuzz4All (LLM)	adapts to new features	LLM on the hot path: slow, costly

Our gap: **valid + replayable + extensible by an LLM, with the LLM off the hot path.**

(Citations to verify before the paper.)

Our idea in one picture

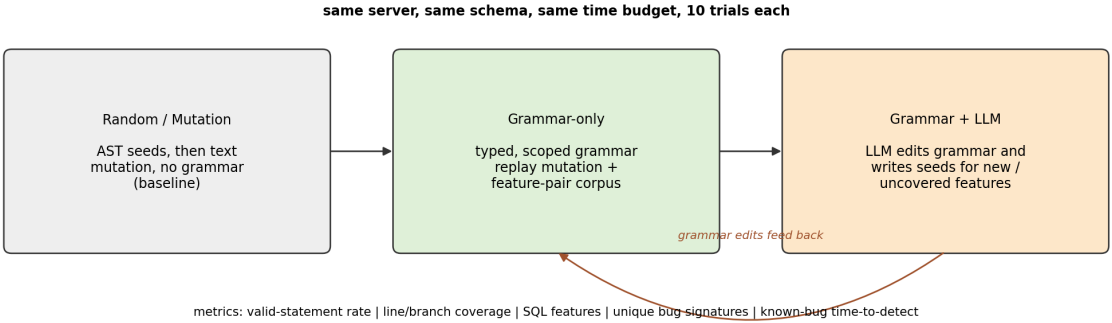


Figure 1: Three arms compared under the same budget, server, schema and oracle

Three claims

1. **One tree, four jobs.** The derivation tree is how we generate, mutate, rewrite (metamorphic oracle) and reduce. SQL text is only the rendering, so a change cannot break types or scopes.
2. **Soundness we measure.** Generator: valid-statement rate. Oracle: a reported mismatch is a real disagreement; triage precision says how often.
3. **The LLM proposes; the server decides.** A proposal enters the fuzzer only if it runs and covers changed lines. That coverage is the proof it helped.

The three-arm comparison stays the experiment. These three claims are what is ours.

Architecture

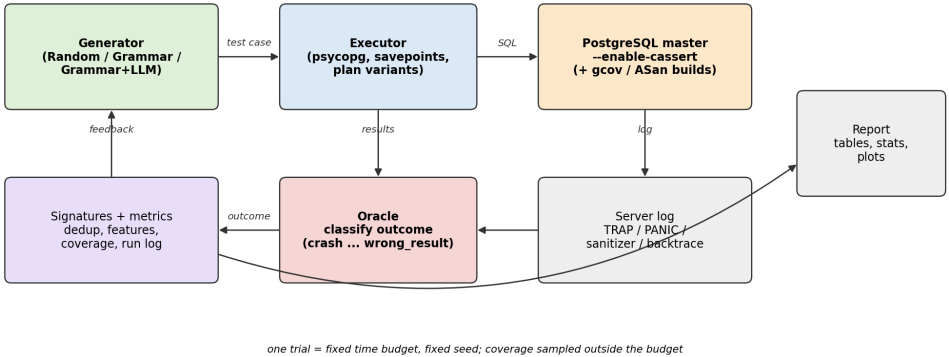
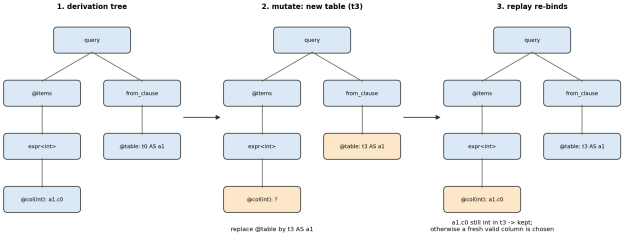


Figure 2: Generator, executor, instrumented PostgreSQL, oracle, signatures, report

Typed grammar and replay mutation

- Our own grammar language `.pgg`: **13 SQL types**, type classes, **scopes**, SQL grouping rules as **context flags** (no aggregates in `WHERE`, group keys only).
- Generation = constructing a typing derivation $\Gamma \vdash e : T$ with backtracking – most statements reach the planner.
- Each statement keeps its **derivation tree**; mutation regenerates or splices a subtree, then **replay re-binds names and types** so the result stays valid.



Oracle stack

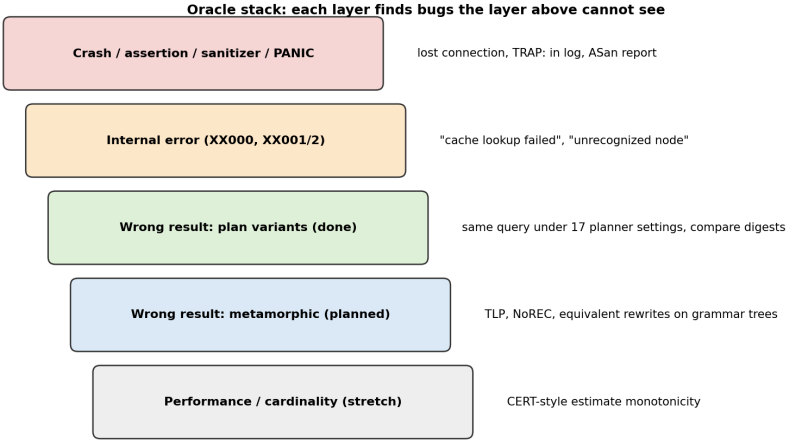


Figure 3: Crash / assertion, internal error, wrong result

The LLM loop (Stage 3, planned)

1. Mine commits on `master` since the last release branch (optimizer, executor, parser).
2. LLM reads message, diff summary, added regression tests and docs.
3. Proposes **grammar edits** and **seed test cases**.
4. **Validation gate:** parses, 200 samples, server validity $>=$ 80 %, new coverage of the **changed lines** – else rejected with feedback.
5. Accepted edits enter the Grammar+LLM arm.

LLM is a *proposal engine*; server and coverage are the *judges*. Metric: **diff coverage** = changed lines executed / changed executable lines.

Experimental design

- **RQ1** Validity and coverage: valid-statement rate, line/branch coverage, plan-shape diversity.
- **RQ2** Bugs: unique crashes, assertion failures, internal errors, wrong results – and how many are real.
- **RQ3** Time to detect: 8–12 **re-injected historical bugs** (revert past fix commits) as ground truth.
- **RQ4** LLM ablation: grammar edits vs seeds vs both; token cost.
- Oracles: crash/assertion, internal error, 17-variant differential, **metamorphic** (TLP, NoREC, tree-level equivalent rewrites).

Statistics and setup

- Same server build (`20devel`, commit `d733fca`, assertions + debug), same schema, executor and oracle for all arms.
- **10 trials per arm**, fixed time budget, different seeds.
- Medians with 95 % CI; **Mann-Whitney U** and **Vargha-Delaney** $\hat{A}_{12}$.
- Coverage collected on a separate gcov build, excluded from the budget.
- Every bug candidate: reproduce, reduce, `git bisect`, check the release, dedup against pgsql-bugs, report upstream.

Status so far (measured)

Item	Result
PostgreSQL from source	`20devel @ d733fca`, assertions, ~13 min on 20 cores
Executor acceptance test	**10/10**
Unit and end-to-end tests	**65** at the end of the harness work; **84** after the grammar and the random baseline
Executor speed, 20 test cases	0.15 s (log read only after a failure) vs 0.99 s
Connection fix on Windows	131 s (`localhost`) -> 0.2 s (`127.0.0.1`)
Grammar	13 types, about 100 rules, exact replay in tests
Random/mutation arm	In the tree: AST seeds, then text mutation, same schema as the grammar

Next: the first multi-hour campaigns. The server measurement is on the next slide.

What the server accepted

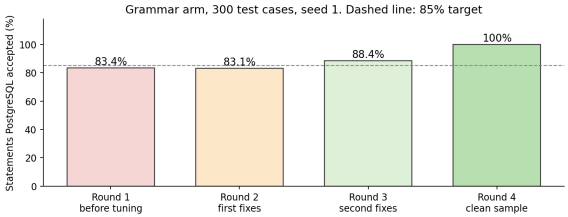


Figure 4: Four rounds of 300 grammar test cases on the same server

- Rounds 1 to 3 climbed from 83.4% to 88.4%. The misses were scopes, JSON values, partitions, and timeouts.
- Round 4: **966 of 966 statements ran (100%)**, in 22.1 seconds. The error list is empty.
- The differential oracle checked 230 queries (460 extra runs). **Still no bug candidate.**
- 100% accepted means every statement reached execution. It does not mean the server is bug-free.

Who does what

- **Venu** and **Bhuvan** – recent PostgreSQL code. Read commits since the last release (MERGE, SQL/JSON, generated columns, partitions, new planner rewrites) and aim the grammar at that code.
- **Bhuvan** – also the first person on any bug candidate a run prints: reproduce it, shrink it, decide if it is real.
- **Gowtham** – the rest of the grade: the comparison experiment, the statistics, the LLM arm, and the paper.

The servers keep running while that reading happens. A candidate does not wait for the paper, and the paper does not wait for a candidate.

Expected outcomes

- **Either way:** RQ1 (validity, coverage), RQ3 (time to detect re-injected bugs) and RQ4 (LLM ablation, token cost) are complete measured results.
- **Target on top of that:** one **new bug the developers acknowledge** (list reply or fix). We file in week 2 so a reply can arrive before 13 Nov.
- **If none is confirmed:** RQ2 reports zero, plus how many candidates we rejected. The motivating example is a re-injected bug with measured times. We do not call that full marks on the 50 % implementation component.
- **Artifacts:** fuzzer, `.pgg` grammar, scripts and the run logs behind every number.

Draft paper structure (FSE style)

Graded parts in the template, and where they sit:

1. Introduction – **motivating example** (a real query, measured numbers)
2. Background
3. Design – **workflow** (one tree; three arms; validation gate)
4. Implementation
5. Evaluation – **research questions** RQ1 to RQ4
6. Bugs found – new bugs, or an honest zero plus oracle precision
7. **Threats to validity** – internal, external, construct
8. **Related work** – each paragraph ends with how we differ
9. Conclusion

Timeline

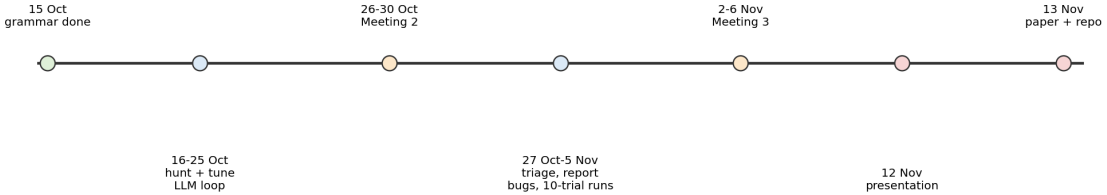


Figure 5: From today to the paper deadline (13 Nov)

Risks and asks

Risk	Mitigation
No new bug in time	file in week 2; RQ1, RQ3, RQ4 and the five graded sections still stand
Upstream slow to respond	report early; a list confirmation is already evidence
False positives	determinism filter, re-runs, manual reasoning
LLM proposals invalid	validation gate; acceptance rate is itself a result

Asks: feedback on the RQs; is an upstream *confirmation* (before a fix commit) acceptable as evidence of a real bug?

Thank you

Questions?